FCSG-Net: Frequency-Band Spatial Expert Routing for Lightweight Composite Image Restoration

Akshat Mittal (123CS0954)
Risav Rathi (123CS0957)
Ramnath Thakur (123CS0965)



Computer Science and Engineering
National Institute of Technology Rourkela

November 2026

FCSG-Net: Frequency-Band Spatial Expert Routing for Lightweight Composite Image Restoration

Project Report submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Akshat Mittal (123CS0954)
Risav Rathi (123CS0957)
Ramnath Thakur (123CS0965)

under the supervision of

Dr. Shyampada Mukherjee



Computer Science and Engineering
National Institute of Technology Rourkela

November 2026

Certificate of Examination

This is to certify that the work in the Project Report entitled *FCSG-Net: Frequency-Band Spatial Expert Routing*
for Lightweight Composite Image Restoration by Akshat Mittal (123CS0954), Risav Rathi (123CS0957) and Ramnath Thakur (123CS0965) has been examined and is accepted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering.

Supervisor	Examiner

Supervisor's Certificate

This is to certify that the work presented in this Project Report entitled *FCSG-Net: Frequency-Band Spatial Expert Routing*
for Lightweight Composite Image Restoration by Akshat Mittal (123CS0954), Risav Rathi (123CS0957) and Ramnath Thakur (123CS0965) is a record of original research carried out under my supervision and guidance in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering. Neither this Project Report nor any part of it has been submitted for any degree or diploma elsewhere.

Dr. Shyampada Mukherjee

Declaration of Originality

We hereby declare that this Project Report is our own work and that, to the best of our knowledge, it contains no material previously published or written by another person, nor material which has been accepted for the award of any other degree, except where due acknowledgement is made in the text. Results reported here are traceable to the project repository and its recorded experiment logs.

Akshat Mittal (123CS0954)
Risav Rathi (123CS0957)
Ramnath Thakur (123CS0965)

Acknowledgement

We are grateful to our supervisor, Dr. Shyampada Mukherjee, for guidance throughout this project.

We thank the Department of Computer Science and Engineering, National Institute of Technology Rourkela, for the opportunity to carry out this work. Our experiments were run on GPUs provided free of charge by Kaggle, without which training four models for 200,000 updates each would not have been possible.

We also thank the authors of DIV2K for making their dataset public, and Kai Zhang and colleagues for releasing the FFDNet code and pretrained weights that let us check our implementation against a published result.

Finally, we thank our families and friends for their patience and support.

Abstract

Archival photographs usually suffer from several degradations at once: optical blur, loss of resolution, sensor or film noise and lossy JPEG compression. Each affects a different part of the image spectrum. This project studies whether a very small restoration network can exploit that structure. The proposed network, FCSG-Net, splits the degraded image into low, mid and high frequency bands with fixed, soft radial masks in the Fourier domain, processes every band with three shared convolutional experts of kernel size 7×7, 5×5 and 3×3, and uses a lightweight gate to select the two most suitable experts for each band at every pixel. The model has 97,088 parameters and costs 9.39 GFLOPs for a 256×256 input. We train FCSG-Net, DnCNN and two FFDNet variants from scratch on the same synthetic composite degradation of DIV2K images, using one schedule and one evaluation protocol, after verifying the implementation and reproducing the published FFDNet result to within 0.01 dB. On 100 validation images FCSG-Net reaches 27.24 dB PSNR, against 27.83 dB for DnCNN and 28.19 dB for blind FFDNet, while using 82.6% fewer parameters than DnCNN. Its low FLOP count does not translate into speed: on a Tesla T4 GPU it is the slowest of the four models. The learned routing is spatially varying and tracks the noise level, with within-image correlations up to 0.81, and overriding the gate's selections at inference costs 0.37 dB on average. Frequency-band routing thus gives a small model inspectable behaviour, but within this budget it does not make the model more accurate or faster than simple convolutional baselines.

Keywords: image restoration, frequency decomposition, mixture of experts, spatial routing, lightweight networks

Contents

Certificate of Examination i

Supervisor's Certificate ii

Declaration of Originality iii

Acknowledgement iv

Abstract v

List of Figures ix

List of Tables xi

List of Abbreviations xii

1 Introduction 1

1.1 Motivation 1

1.2 Problem Statement 2

1.3 Research Hypothesis and Objectives 2

1.4 Contributions 3

1.5 Organisation of the Report 3

2 Background and Related Work 4

2.1 Image Restoration with CNNs 4

2.2 Frequency-Domain Restoration 5

2.3 Mixture of Experts and Gating 6

2.4 Evaluation Metrics 6

2.5 Research Gap 7

3 Proposed Method: FCSG-Net 8

3.1 Degradation Model 8

3.2 Architecture Overview 9

3.3 Frequency Decomposition 10

3.4 Shared Kernel-Size Experts . . . 11
3.5 Spatial Top-2 Gate . . . 11
3.6 Fusion and Residual Refinement . . . 13
3.7 Training Objective . . . 13
3.8 Model Size and Complexity . . . 14

4 Experimental Setup and Verification **15**
4.1 Dataset and Data Pipeline . . . 15
4.2 Baselines . . . 16
4.3 Training Protocol . . . 17
4.4 Evaluation Protocol . . . 18
4.5 Implementation Verification . . . 18
4.6 Baseline Reproduction . . . 19

5 Restoration Results and Computational Cost **20**
5.1 Quantitative Comparison . . . 20
5.2 Per-Image Analysis . . . 22
5.3 Qualitative Results . . . 23
5.4 Training Convergence . . . 23
5.5 Latency, Memory and Tiling . . . 23

6 Routing Analysis **29**
6.1 Training Dynamics of the Gate . . . 29
6.2 Spatial Routing Statistics . . . 30
6.3 Routing Versus Degradation Parameters . . . 31
6.4 Inference-Time Interventions . . . 32
6.5 Discussion . . . 33

7 Conclusion and Future Work **34**
7.1 Summary of Findings . . . 34
7.2 Limitations . . . 35
7.3 Future Work . . . 35

References **37**

A Reproducibility Record **39**
A.1 Code and Environment . . . 39
A.2 Where Each Result Comes From . . . 39
A.3 Artefact Identifiers . . . 40
A.4 How to Reproduce . . . 40

B Per-Image Results **41**

List of Figures

3.1 FCSG-Net data flow. Expert parameters are shared across the three bands; the band dimension is folded into the batch dimension before each expert call. 9
3.2 Raised-cosine band masks with cutoffs 0.10 and 0.25 cycles per pixel and transition width 0.05. Right: the masks on a centred frequency plane (the computation itself uses unshifted ordering). 11
3.3 Expert and gate operations. Experts use pixel rearrangement and depthwise-separable residual blocks; the gate pools band statistics in 16×16 windows. 12

4.1 Training data pipeline. Loader workers sample degradation parameters for each augmented clean crop and return the degraded input, the clean target and the parameters. 16
4.2 Learning-rate and entropy-coefficient schedules over 200,000 updates. 17
4.3 Single-image overfit test: MSE over 200 updates. 19

5.1 Mean PSNR, SSIM and LPIPS over the 100 validation images. Each bar is one training run. 21
5.2 Left: per-image PSNR difference between FCSG-Net and each baseline. Right: sorted per-image RMSE for every model (in 8-bit units). . . . 22
5.3 Degraded input, restoration and clean target for FCSG-Net and DnCNN. 24
5.4 Degraded input, restoration and clean target for the two FFDNet variants. 25
5.5 Validation PSNR on 16 fixed crops against updates and training time. Crop PSNR is not comparable with the full-image results. 26
5.6 Validation PSNR against reported GFLOPs (left) and measured 256×256 forward latency (right). 27
5.7 Tile-size sensitivity: PSNR change relative to 256-pixel tiles, and the tile and overlap geometry. 28

6.1 Dense routing entropy summed over bands during training. The maximum is $3 \ln 3 \approx 3.296$ nats. 29

6.2 Mean sparse weight and selection fraction for each expert and band over 80 samples. 30
6.3 Expert weights for validation image 0834 (seed 33). Rows: low, mid and high band. Columns: degraded crop and the weights of the 7×7, 5×5 and 3×3 experts, on a common 0 to 1 scale. 31
6.4 Left: within-image Pearson correlations between mean routing weights and degradation parameters. Right: per-image PSNR change under the inference interventions of Table 6.1. 32

List of Tables

2.1 Positioning of FCSG-Net against closely related methods. 7

3.1 Composite degradation pipeline. Parameters are sampled independently for every image. 8
3.2 FCSG-Net parameter breakdown. 14

4.1 Training configuration, identical for every method. 17
4.2 Pre-training verification checks (Tesla T4, 25 September 2026 run). . 18
4.3 DnCNN on clipped Gaussian noise, $\sigma = 25$, checkpoint 286,000. . . . 19

5.1 Composite restoration on 100 DIV2K validation images after 200,000 updates. Gain is output minus input PSNR (input 20.992 dB). GFLOPs at 256×256. 20
5.2 FCSG-Net relative to each baseline. 21
5.3 Per-image PSNR comparison of FCSG-Net against each baseline, 100 images. 22
5.4 Median forward latency (ms) by input size and peak inference memory at 256×256 (Tesla T4, FP32, batch 1). 26
5.5 Mean PSNR (dB) on ten validation images for three tile sizes, 32-pixel overlap. 27

6.1 PSNR change when the trained gate is overridden at inference (10 images). 32

A.1 Source files for the results in this report. 39
A.2 SHA-256 identifiers (first 16 hex digits) of evaluated artefacts. 40

B.1 Per-image PSNR (dB) on DIV2K validation. 41

List of Abbreviations

AMP	Automatic Mixed Precision
AWGN	Additive White Gaussian Noise
CNN	Convolutional Neural Network
DIV2K	DIVerse 2K resolution image dataset
FFT	Fast Fourier Transform
FLOP	Floating-Point Operation
GFLOP	10^9 Floating-Point Operations
GPU	Graphics Processing Unit
JPEG	Joint Photographic Experts Group (compression)
LPIPS	Learned Perceptual Image Patch Similarity
MAC	Multiply-Accumulate Operation
MoE	Mixture of Experts
MSE	Mean Squared Error
PSNR	Peak Signal-to-Noise Ratio
ReLU	Rectified Linear Unit
RMSE	Root Mean Squared Error
SE	Squeeze-and-Excitation
SSIM	Structural Similarity Index

Chapter 1

Introduction

1.1 Motivation

Old photographs rarely suffer from a single, clean corruption. A print that was scanned at modest resolution, stored as a heavily compressed JPEG and captured in the first place through a soft lens on grainy film carries all of those defects at once. Each of them leaves its mark on a different part of the image spectrum. Optical softening removes high spatial frequencies. Resampling discards detail above a resolution limit and introduces interpolation artefacts. Sensor and film noise spreads energy roughly evenly across frequencies, so it dominates wherever the true signal is weak, which in natural images means the high frequencies. Block-based JPEG compression quantises the mid and high frequency coefficients of each 8×8 block and leaves blocking and ringing behind.

Most learned restoration networks treat this mixture as one problem. A deep convolutional network such as DnCNN [1] sees the whole degraded image and learns, implicitly, how to undo every corruption in it. Larger transformer-based restorers such as Restormer [2] are very effective, but their tens of millions of parameters put them out of reach for the low-cost hardware that an archive, a library or a hobbyist is likely to own.

This project starts from a simple observation. If different corruptions live in different frequency bands, it might be useful to split the image into bands explicitly and let a small network decide, for every band and every location, which kind of filter suits it best. A large kernel can follow smooth, slowly varying structure. A small kernel can respond to fine detail and to the high-frequency residue of noise. A learned gate can choose between them pixel by pixel. We call the resulting network FCSG-Net, a frequency-conditioned, spatially gated network, and we study it under a strict budget: fewer than five million parameters and fewer than ten GFLOPs for a 256×256 input.

1.2 Problem Statement

Let $f \in [0,1]^{H \times W \times 3}$ be a clean RGB image and let

$$x = \mathcal{D}(f;\, \theta), \qquad \theta = (\sigma_{\text{blur}},\, s,\, \sigma_n,\, q) \tag{1.1}$$

be a degraded observation produced by Gaussian blur of width σ_{blur}, bicubic down- and up-sampling by a factor s, additive Gaussian noise of standard deviation σ_n and JPEG compression at quality q. The parameters θ are drawn at random for every image and are not given to the restoration model. The task is to learn a mapping g_ϕ such that $\hat{f} = g_\phi(x)$ is close to f, measured by PSNR, SSIM and LPIPS, subject to two hard resource limits:

- the network has fewer than 5×10^6 trainable parameters, and
- one forward pass on a 256×256 RGB input costs fewer than 10 reported GFLOPs.

1.3 Research Hypothesis and Objectives

The working hypothesis of the project is the following. *Spatial top-2 routing over shared convolutional experts of different kernel sizes, applied after a fixed radial frequency decomposition, yields a routing policy that varies with image content and with the degradation, and that the trained network relies on for its output quality, while staying within the stated budget.* We deliberately phrase this as something to be tested rather than as a claim of superiority.

The work is organised around four objectives:

O1 Build FCSG-Net and verify it: exact band reconstruction, parameter and FLOP limits, memory, gradient flow and the ability to overfit a single image (Chapters 3 and 4).

O2 Establish trustworthy baselines by reproducing published FFDNet behaviour and training DnCNN on a known Gaussian task (Chapter 4).

O3 Train FCSG-Net, DnCNN and two FFDNet variants under one matched composite degradation protocol and compare quality, size, compute and measured latency (Chapter 5).

O4 Characterise the learned routing: where it sends each band, how it relates to the sampled degradation, and how much the trained model depends on it (Chapter 6).

1.4 Contributions

The contributions of this report are practical and specific:

- A frequency-band, spatially routed restoration network with 97,088 parameters and 9.39 reported GFLOPs at 256×256, together with the verification tests that establish that it does what its equations say.
- A matched comparison in which DnCNN, blind FFDNet, oracle FFDNet and FCSG-Net are all trained from scratch on the same synthetic composite degradation, with the same schedule, the same number of updates and the same evaluation protocol.
- A routing analysis that relates per-pixel expert weights to the recorded degradation parameters and measures the effect of overriding the gate at inference time.
- An honest account of the outcome. FCSG-Net reaches 27.24 dB against 27.83 dB for DnCNN and 28.19 dB for blind FFDNet with 5.7 times fewer parameters than DnCNN, but it is not faster on a GPU, because a low FLOP count did not translate into low latency.

We do not claim state-of-the-art restoration quality, and we do not claim that frequency decomposition or expert routing are new ideas. Both are well established in prior work (Chapter 2). What we test is one particular combination of them under a tight budget.

1.5 Organisation of the Report

Chapter 2 reviews restoration networks, frequency-domain methods and mixture-of-experts routing, and positions FCSG-Net among them. Chapter 3 describes the degradation model and the network, module by module. Chapter 4 covers the data pipeline, the baselines, the training and evaluation protocols and the verification experiments. Chapter 5 reports restoration quality and computational cost. Chapter 6 analyses the learned routing. Chapter 7 summarises the findings and their limitations and lists the experiments that should come next. Appendix A records what is needed to reproduce the work, and Appendix B lists per-image results.

Chapter 2

Background and Related Work

This chapter places FCSG-Net among three lines of work: convolutional restoration networks, restoration in the frequency domain, and mixture-of-experts routing. Published scores from these papers come from different tasks, datasets and noise protocols. We quote them only as context. None of them appear in our result tables, which contain only models we trained and evaluated ourselves under one protocol.

2.1 Image Restoration with CNNs

DnCNN. Zhang et al. [1] showed that a plain stack of convolutions, batch normalisation and ReLU learns denoising far more easily when it predicts the noise residual rather than the clean image. The output is formed by subtracting the predicted residual from the input. Our RGB implementation has 17 layers of width 64 and 558,403 parameters. DnCNN has no explicit noise-level input and no notion of frequency bands, which makes it a clean reference for what a conventional network achieves on our data. The original paper also studies blind denoising and several restoration tasks, so it is not limited to one fixed noise level.

FFDNet. FFDNet [3] takes two ideas further. It rearranges the input into four half-resolution sub-images before convolution, which cuts computation by a factor of about four, and it concatenates a noise-level map to those sub-images so that a single network can handle a range of noise strengths. The colour model has 12 layers of width 96. The author's KAIR inference release [4] has 852,108 parameters, with batch normalisation folded into the convolutions. The paper reports 31.21 dB on CBSD68 at $\sigma = 25$ using unclipped Gaussian noise. FFDNet is our second baseline. Its noise map, however, only describes Gaussian noise and says nothing about blur or JPEG artefacts.

Larger restorers. NAFNet [5] shows that carefully simplified blocks without conventional nonlinear activations can match far more complex designs, which is a reminder that architectural novelty alone is weak evidence of progress. Restormer [2] represents the heavy end: transposed channel attention and gated feed-forward blocks with about 26 million parameters in its deblurring configuration. Both sit outside our budget and serve as context rather than as competitors.

2.2 Frequency-Domain Restoration

MWCNN. Liu et al. [6] replace pooling and upsampling in a U-Net with discrete wavelet and inverse wavelet transforms. The transforms are fixed and add no parameters, so the convolutional backbone carries the model's capacity. Importantly, MWCNN processes subbands jointly and its ablations argue against handling each subband separately. That result motivates the cross-band fusion stage in FCSG-Net. MWCNN establishes that explicit frequency decomposition is useful for restoration. It does not route subbands to different kernel sizes.

Fast Fourier Convolution. FFC [7] splits channels between a local convolutional branch and a global branch that operates in the Fourier domain, which gives every layer an image-wide receptive field. FFC is an operator, not a single restoration model, and its cost depends on the backbone it is inserted into. FCSG-Net differs in that it returns each masked band to the spatial domain before any learned processing.

SFNet and FSNet. The closest prior work is by Cui et al. SFNet [8] uses content-dependent local decomposition with channel attention to select frequency information, together with a module that pools globally and in windows to widen the receptive field cheaply. FSNet [9] extends the approach to six restoration tasks with multiple receptive-field scales in one U-shaped network. Both already perform local, adaptive frequency selection. The distinction we test is narrower: FCSG-Net uses *fixed* radial masks and puts the learned, spatial decision in the choice among shared 7×7, 5×5 and 3×3 experts, which we can then compare with known degradation parameters. Fixed bands on their own cannot be claimed as an advance over SFNet or FSNet.

FFTformer. Kong et al. [10] compute attention-related operations in the frequency domain for deblurring, with 16.6 million parameters in their GoPro model. Their use of local Fourier processing also shows that the transform's spatial context is part of the architecture, which is relevant to our tiled inference (Section 5.5).

Spectral bias. Rahaman et al. [11] show that ReLU networks tend to fit low-frequency components of a target function before high-frequency ones. This motivates looking at errors band by band, but it does not prove that a low band needs a large kernel or that separate experts will specialise. Those remain empirical questions.

2.3 Mixture of Experts and Gating

Shazeer et al. [12] introduced the sparsely gated mixture-of-experts layer, in which a noisy top-k gate selects a small subset of experts for each example and auxiliary importance and load penalties prevent the gate from collapsing onto a few experts. Those penalties balance usage across a batch. They are not the same as maximising the routing entropy of each example, which would push every location towards uniform weights and work against specialisation. Fedus et al. [13] simplify routing to a single expert per token in the Switch Transformer and discuss capacity, load balancing and numerical stability at very large scale.

Two points from this literature shape our design and our claims. First, the FCSG-Net gate is spatial: it produces a routing map over pixels rather than a single decision per image. Second, the efficiency argument for sparse experts depends on *conditional computation*, that is, not running the experts that receive zero weight. Our implementation computes every expert output and then mixes them with sparse weights. The sparsity is therefore a property of the weights and of the analysis, not a source of speed, and we account for the full cost of all experts in every FLOP and latency figure.

The fusion stage uses a squeeze-and-excitation block [14], which pools features globally, compresses them through a small bottleneck and predicts per-channel scales. In FCSG-Net it reweights the nine band channels before projecting them back to RGB, at a cost of 96 parameters.

2.4 Evaluation Metrics

PSNR. For an image with pixel values in $[0, 1]$, the peak signal-to-noise ratio is

$$\mathrm{PSNR}(\hat{f}, f) = 10 \log_{10} \frac{1}{\mathrm{MSE}(\hat{f}, f)}, \qquad \mathrm{MSE} = \frac{1}{3HW} \sum_{c,u,v} \big(\hat{f}_{c,u,v} - f_{c,u,v}\big)^2. \tag{2.1}$$

We compute PSNR per image over all RGB channels and report the arithmetic mean over the dataset.

SSIM. The structural similarity index [15] compares local means, variances and covariances within a sliding window. We use an 11×11 Gaussian window with $\sigma = 1.5$, constants $C_1 = 0.01^2$ and $C_2 = 0.03^2$, and average over channels and positions.

LPIPS. LPIPS [16] measures the distance between normalised deep features of two images, with channel weights calibrated against human perceptual judgements. We use the AlexNet variant with inputs scaled to $[-1, 1]$. Lower is better. LPIPS can disagree with PSNR because perceptual similarity and exact pixel agreement are different properties. The metric network is an evaluation tool and is not counted in any model's parameters.

2.5 Research Gap

Table 2.1 summarises how FCSG-Net relates to the methods above. The combination it tests is fixed frequency bands, shared kernel-size experts, per-pixel top-2 routing and a budget below 0.1 million parameters. No single prior method we reviewed occupies exactly this point, but each ingredient is established, so the contribution of this project is an empirical study of the combination rather than a new building block.

Table 2.1: Positioning of FCSG-Net against closely related methods.

Method	Freq. split	Learned selection	Spatial	Notes
DnCNN [1]	–	–	–	residual CNN
FFDNet [3]	–	–	–	noise-map input
MWCNN [6]	wavelet	–	–	joint subbands
SFNet [8]	learned	frequency	local	closest prior work
Sparse MoE [12]	–	experts	–	per-example routing
FCSG-Net (ours)	radial FFT	experts, top-2	per pixel	97k parameters

Chapter 3

Proposed Method: FCSG-Net

This chapter describes the synthetic degradation used for training and evaluation, and then the network itself, one module at a time. The notation follows the implementation in `models/fcsg.py` and `models/blocks.py`.

3.1 Degradation Model

Archival photographs are degraded in a rough physical order: the lens softens the image, scanning or resizing loses resolution, the sensor or film adds noise, and lossy storage compresses whatever is left. Our synthetic pipeline applies the four operations in that order (Table 3.1).

Table 3.1: Composite degradation pipeline. Parameters are sampled independently for every image.

Stage	Parameter	Distribution
Gaussian blur	σ_{blur}	$\mathcal{U}(0.5, 2.0)$ pixels
Down/up resampling	scale s	$\mathcal{U}(1.0, 2.0)$, bicubic both ways
Additive noise	σ_n	$\mathcal{U}(5, 50)/255$, clipped to $[0, 1]$
JPEG compression	quality q	integer, uniform on $\{30, \ldots, 95\}$, 4:4:4

Blur uses Pillow's Gaussian filter. Resampling shrinks the image by s with bicubic interpolation and enlarges it back to the original size, so input and output always have the same dimensions. Noise is additive white Gaussian noise in 8-bit units, clipped to the valid range. JPEG encoding uses Pillow with chroma subsampling disabled. The image is quantised to 8 bits before the Pillow blur and before JPEG encoding, as a real file would be.

Each sample carries its four sampled parameters alongside the degraded and clean images. The network never sees them. They are recorded so that, after training, we can ask whether the router responds to the degradation (Chapter 6). With noise

between 5 and 50 grey levels and JPEG quality as low as 30, the mean input PSNR on our validation set is 20.99 dB.

3.2 Architecture Overview

Figure 3.1 shows the data flow. The degraded image x is split into a low, a mid and a high frequency band, each of which is an ordinary RGB image. Three shared experts with 7×7, 5×5 and 3×3 kernels process every band. A spatial gate assigns, at each pixel and for each band, a weight to every expert, keeping only the two largest. The weighted expert outputs form one restored image per band:

$$y_b(u, v) = \sum_{e \in \{7,5,3\}} w_{b,e}(u, v)\; E_e(x_b)(u, v), \qquad b \in \{L, M, H\}. \tag{3.1}$$

The three band outputs are concatenated into nine channels, reweighted by a squeeze-and-excitation block, projected back to RGB and refined. The network predicts a residual that is added to its input:

$$\hat{f} = x + \mathrm{Refine}\big(\mathrm{SEFusion}([\, y_L, y_M, y_H \,])\big). \tag{3.2}$$

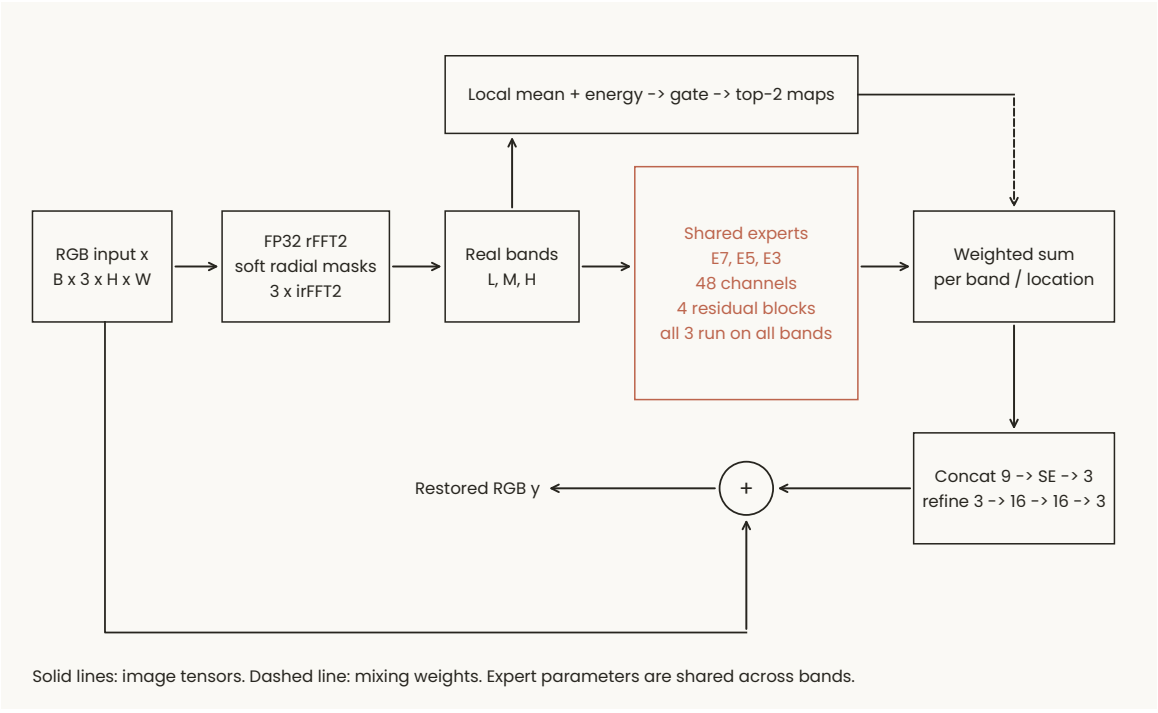


Figure 3.1: FCSG-Net data flow. Expert parameters are shared across the three bands; the band dimension is folded into the batch dimension before each expert call.

3.3 Frequency Decomposition

The decomposition is done in the Fourier domain with fixed masks, but the result of each mask is returned to the spatial domain immediately. We made this choice early and deliberately. An earlier version of the proposal fed masked complex spectra straight into the convolutional experts. That has two problems. A convolution over a spectrum has no clean meaning, since multiplication in one domain corresponds to convolution in the other, so the argument that large kernels suit smooth structure no longer applies. And complex inputs either become two real channels, where phase is discontinuous and training becomes unstable, or require complex tensor support that many PyTorch operations lack. Masking in frequency and filtering in space keeps the decomposition exact and lets every expert be a plain real-valued CNN.

Let $X = \mathcal{F}(x)$ be the two-dimensional real FFT of the image with orthonormal scaling, and let $r = \sqrt{f_x^2 + f_y^2}$ be the radial frequency in cycles per pixel. Frequencies are built with `fftfreq` and `rfftfreq` in the unshifted order the FFT uses. Each axis reaches the Nyquist limit at 0.5 cycles per pixel. A soft low-pass mask with cutoff c and transition width $\Delta = 0.05$ is a raised cosine:

$$t_c(r) = \mathrm{clip}\left(\frac{r - c}{\Delta} + \tfrac{1}{2},\ 0,\ 1\right), \qquad \mathrm{LP}_c(r) = \tfrac{1}{2}\big(1 + \cos(\pi\, t_c(r))\big). \tag{3.3}$$

With cutoffs $r_1 = 0.10$ and $r_2 = 0.25$, the three band masks are

$$M_L = \mathrm{LP}_{r_1}, \qquad M_M = \mathrm{LP}_{r_2} - \mathrm{LP}_{r_1}, \qquad M_H = 1 - \mathrm{LP}_{r_2}, \tag{3.4}$$

and each band image is $x_b = \mathcal{F}^{-1}(M_b \cdot X)$.

Three properties matter. First, the masks sum to one at every frequency, and the FFT is linear, so $x_L + x_M + x_H = x$ exactly; no information is lost or invented by the split. We measured a maximum reconstruction error of 6.0×10^{-7} in single precision. Second, the cutoffs are defined in cycles per pixel rather than in FFT bin indices. A bin index means a different physical frequency at 128×128 than at full resolution, so bin-based cutoffs would shift between training crops and test images. With normalised radii the same sinusoid falls in the same band at both sizes, which we verified at 128 and 256 pixels. Third, the transitions are smooth. A hard step in frequency produces ringing in space, and the network would spend capacity undoing artefacts of its own preprocessing.

The FFT runs in single precision even during mixed-precision training, because half-precision CUDA FFTs only support power-of-two sizes. Figure 3.2 plots the three masks.

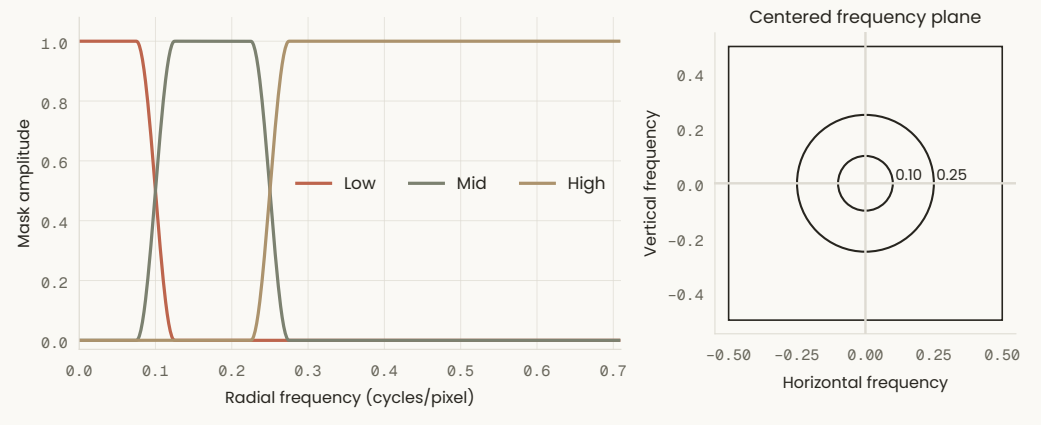


Figure 3.2: Raised-cosine band masks with cutoffs 0.10 and 0.25 cycles per pixel and transition width 0.05. Right: the masks on a centred frequency plane (the computation itself uses unshifted ordering).

3.4 Shared Kernel-Size Experts

There are three experts, E_7, E_5 and E_3, identical except for their kernel size. Each one processes all three bands, so the parameters are shared across bands and the number of experts does not grow with the number of bands.

An expert first applies a lossless 2×2 pixel unshuffle [17], turning a three-channel image into twelve channels at half resolution, as FFDNet does. A 1×1 convolution expands these to 48 channels, followed by ReLU. Four residual blocks follow. Each block contains two depthwise-separable stages [18]: a $k \times k$ depthwise convolution and a 1×1 pointwise convolution, with a ReLU after the first stage. A 1×1 tail maps back to twelve channels and is added to the unshuffled input, and a pixel shuffle restores the original resolution. Odd image sizes are padded by replication and cropped back.

Separable convolutions and half-resolution processing are what make the budget possible. The experts run nine times per image, once per band for each of three experts, and dense full-resolution convolutions at width 48 would exceed ten GFLOPs by a wide margin.

We refer to the experts only by kernel size. Labels such as "texture expert" or "edge expert" would be claims about what the experts learned, and we have no attribution experiment that supports them.

3.5 Spatial Top-2 Gate

A routing decision that is made once per image would amount to nine numbers computed from a handful of global statistics. That is closer to a lookup table than to a router, and it is likely to settle on a near-constant choice. The FCSG-Net gate instead decides per location, so that a smooth sky and a textured roof in the same

photograph can be treated differently.

The gate stacks the three RGB bands into nine channels and computes, in non-overlapping 16×16 windows, the local mean and the local mean of squares of each channel. These eighteen statistics pass through a two-layer 1×1 network ($18 \to 24 \to 9$, ReLU in between) that outputs nine logits $a_{b,e}$, one per band and expert. The logits are divided by a temperature $\tau = 1$ and bilinearly upsampled to full resolution. At each pixel and for each band the two largest logits are kept and normalised with a softmax; the third expert gets zero weight:

$$w_{b,e}(u,v) = \begin{cases} \dfrac{\exp a_{b,e}(u,v)}{\sum_{j \in \mathcal{T}_b(u,v)} \exp a_{b,j}(u,v)} & e \in \mathcal{T}_b(u,v), \\ 0 & \text{otherwise}, \end{cases} \tag{3.5}$$

where $\mathcal{T}_b(u,v)$ is the set of the two highest-scoring experts. The gate also returns the dense softmax over all three experts, $p_{b,e}$, which is used by the entropy term of the loss. The whole gate has 681 parameters. Figure 3.3 shows the expert and gate operations.

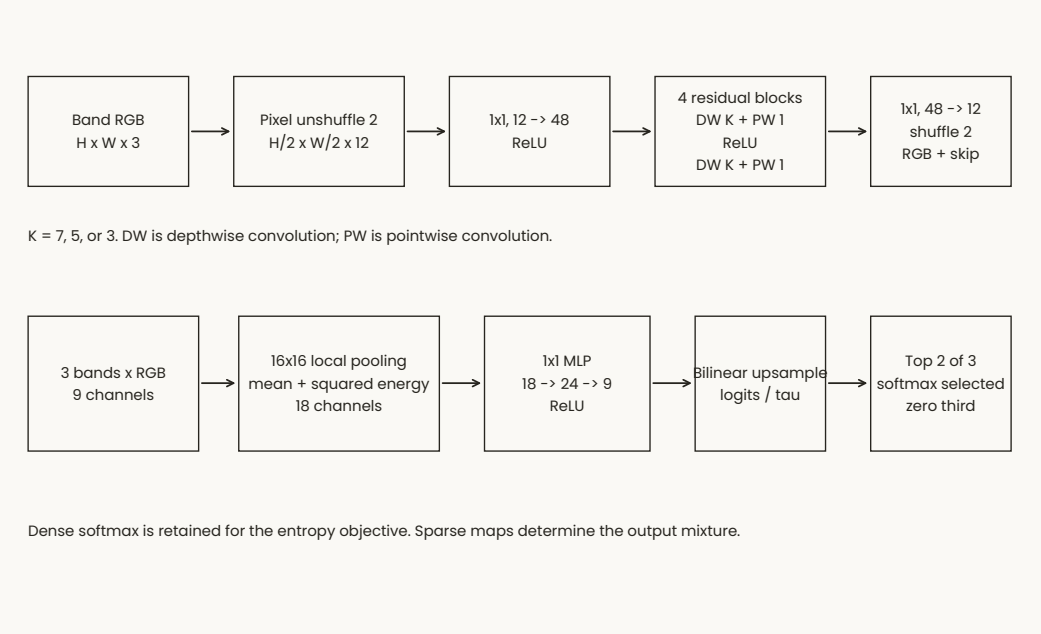


Figure 3.3: Expert and gate operations. Experts use pixel rearrangement and depthwise-separable residual blocks; the gate pools band statistics in 16×16 windows.

As noted in Section 2.3, every expert is evaluated before mixing. Expert outputs at locations with zero weight are computed and then multiplied by zero. Turning the sparse weights into saved computation would require tiled conditional dispatch, which we did not implement.

3.6 Fusion and Residual Refinement

The three weighted band outputs are concatenated into nine channels. A squeeze-and-excitation block [14] pools them globally, passes them through a $9 \to 3 \to 9$ bottleneck with a sigmoid, and scales each channel. A 1×1 convolution projects the result to three channels. Three 3×3 convolutions ($3 \to 16 \to 16 \to 3$, ReLU between them) refine it, and the output is added to the degraded input as in Equation (3.2). Predicting a residual rather than the image itself is the central lesson of DnCNN [1], and it lets the network start from the identity.

3.7 Training Objective

The loss has three terms:

$$\mathcal{L} = \mathcal{L}_{\text{char}} + 0.05\,\mathcal{L}_{\text{freq}} + \lambda_{\text{ent}}(t)\,\mathcal{L}_{\text{ent}}. \tag{3.6}$$

The Charbonnier loss [19] is a smooth variant of L1,

$$\mathcal{L}_{\text{char}} = \text{mean}\sqrt{(\hat{f} - f)^2 + \varepsilon^2}, \qquad \varepsilon = 10^{-3}, \tag{3.7}$$

which behaves like L2 near zero and like L1 for large errors.

The frequency term is the mean absolute value of the orthonormal FFT of the prediction error, $\mathcal{L}_{\text{freq}} = \text{mean}|\mathcal{F}(\hat{f} - f)|$. It weights errors at every frequency equally, which counteracts the tendency of pixel losses to favour the dominant low frequencies.

The entropy term is the negative dense routing entropy, summed over bands and averaged over the batch and pixels:

$$\mathcal{L}_{\text{ent}} = \sum_{b} \text{mean}_{n,u,v} \sum_{e} p_{b,e} \log p_{b,e}. \tag{3.8}$$

Minimising it maximises entropy, which keeps all three experts in use early in training so that none of them is starved of gradient. A constant entropy bonus would, however, push the router towards uniform weights forever, which is the opposite of what we want to observe. The coefficient therefore decays linearly, $\lambda_{\text{ent}}(t) = 0.01\,(1 - t/T)$, reaching zero at the last update T. Because the sign of this term is easy to flip by accident, the verification suite checks that it decreases as the weights approach uniform. DnCNN and FFDNet are trained with the same Charbonnier and frequency terms and no entropy term.

3.8 Model Size and Complexity

Table 3.2 breaks down the 97,088 parameters. Almost all of them are in the three experts; the gate, fusion and refinement stages together account for under four thousand.

Table 3.2: FCSG-Net parameter breakdown.

Module	Parameters
Shared 7×7 expert	39,228
Shared 5×5 expert	30,012
Shared 3×3 expert	23,868
Spatial gate ($18 \to 24 \to 9$)	681
SE fusion and $9 \to 3$ projection	96
Residual refinement	3,203
Total	97,088

For compute we profile multiply-accumulate operations (MACs) with `thop`, count each MAC as two floating-point operations, and add an estimate of $12 \times 5N \log_2 N$ for the forward and inverse FFTs, with $N = 256^2$. Small elementwise operations are not counted. Under this convention a 256×256 forward pass costs 4.66 GMACs, or 9.39 GFLOPs including the FFT estimate, inside the 10 GFLOP limit. The same convention is applied to the baselines. As Chapter 5 shows, this count turned out to be a poor predictor of actual running time.

Chapter 4

Experimental Setup and Verification

The numbers in Chapter 5 are only as good as the code and protocol behind them. This chapter describes the data, the baselines and how every model was trained and evaluated, and then the tests we ran before committing GPU time to long training runs.

4.1 Dataset and Data Pipeline

All experiments use DIV2K [20], which provides 800 high-resolution training images and 100 validation images. Training uses only the training split; the 100 validation images are used for every result we report. Because we did not hold out a separate test set, the reported numbers are validation results, a limitation we return to in Chapter 7.

Our first data loader decoded a full-resolution PNG for every crop, and training ran at roughly 1.8 steps per second because the GPU waited on the CPU. We replaced it with a cache of 32 fixed 128×128 crops from each training image, 25,600 tiles in all, stored as a single 1.26 GB array of 8-bit pixels and memory-mapped at training time. Random flips and 90-degree rotations augment the cached crops. The degradation of Section 3.1 is still applied on the fly by four loader workers, so every epoch sees freshly sampled blur, scale, noise and JPEG settings. In effect the set of degraded pairs is unlimited, while the set of clean crops is fixed (Figure 4.1).

Online degradation is good for training but awkward for anyone trying to reproduce exact inputs. We therefore also exported a fixed set of 5,000 degraded and clean pairs generated with seed 1234, together with a manifest of their parameters. The degradation code reproduces this set byte for byte, and a single CPU worker generates about 233 samples per second.

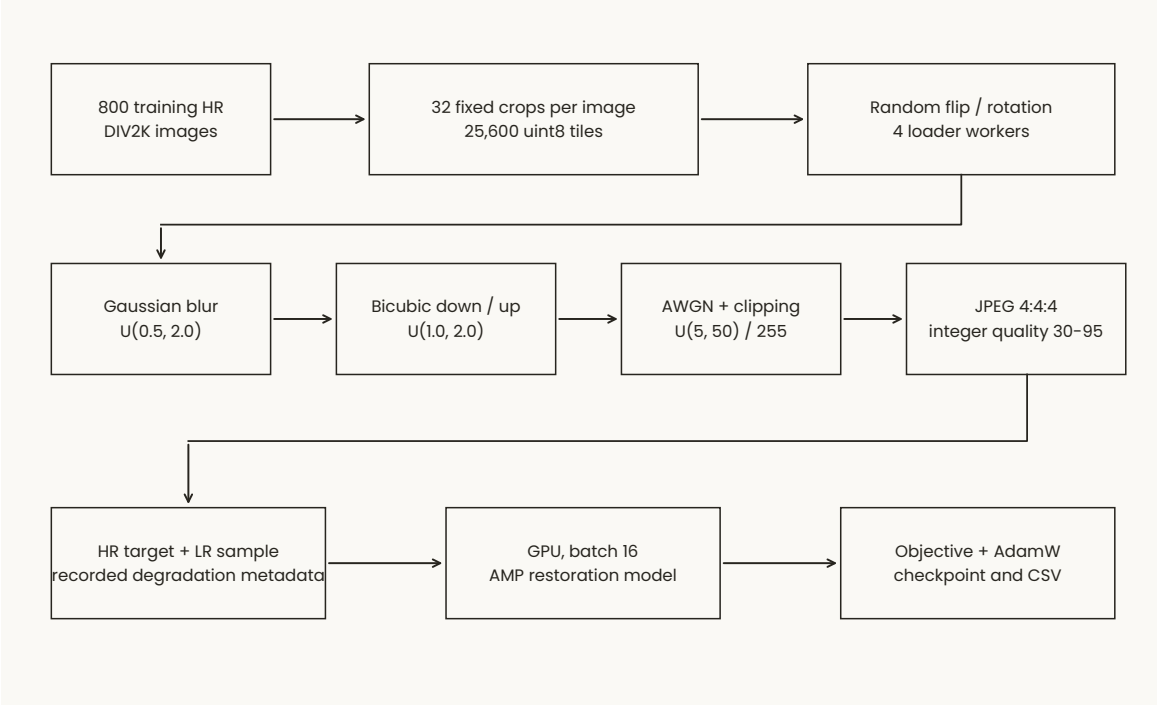


Figure 4.1: Training data pipeline. Loader workers sample degradation parameters for each augmented clean crop and return the degraded input, the clean target and the parameters.

4.2 Baselines

DnCNN. Our RGB DnCNN has 17 convolutional layers of width 64 with batch normalisation and predicts the residual, giving 558,403 parameters.

FFDNet. Our FFDNet follows the author's colour architecture [4]: pixel unshuffle, 12 convolutional layers of width 96 and a noise-level map concatenated to the input, giving 852,108 parameters. On our composite task the noise map is a problem, because there is more to the degradation than Gaussian noise. We therefore train two variants. *Oracle* FFDNet receives the true sampled noise level σ_n as its map. This is information no real user would have, so the oracle variant is an upper reference rather than a fair competitor. *Blind* FFDNet receives a constant map of 27.5/255, the middle of the noise range, during both training and evaluation. It sees exactly what DnCNN and FCSG-Net see.

All four models are trained from scratch on the same composite degradation. We do not compare against numbers published for other tasks or datasets.

4.3 Training Protocol

Every model is trained with the same configuration (Table 4.1), differing only in the model and, for FFDNet, the noise map. The original plan called for 300,000 updates; we reduced this to 200,000 to fit the weekly GPU quota available to us.

Table 4.1: Training configuration, identical for every method.

Setting	Value
Optimiser	AdamW [21], weight decay 0
Learning rate	2×10^{-4}, cosine decay [22] to 10^{-6}
Updates	200,000
Batch	16 crops of 128×128 RGB
Loss	$\mathcal{L}_{\text{char}} + 0.05\,\mathcal{L}_{\text{freq}}$, plus entropy term for FCSG-Net
Entropy coefficient	0.01, linear decay to 0 (FCSG-Net only)
Precision	FP16 mixed precision [23] with gradient scaling
Validation	16 fixed crops every 2,000 updates
Checkpoints	every 2,000 updates (model, optimiser, scaler, step)
Hardware	NVIDIA Tesla T4 (Kaggle)

Training ran in Kaggle notebook sessions, which stop after twelve hours. Each run therefore resumes from its newest checkpoint at start-up. Checkpoints store the model, optimiser, gradient scaler, step and configuration. They do not store random number generator or sampler state, so a resumed run is not bit-identical to an uninterrupted one. Training seeds were not fixed, and each model was trained once. Recorded training-loop time was 10.06 hours for FCSG-Net, 6.66 for DnCNN, 2.04 for blind FFDNet and 2.33 for oracle FFDNet. Figure 4.2 shows the learning-rate and entropy schedules.

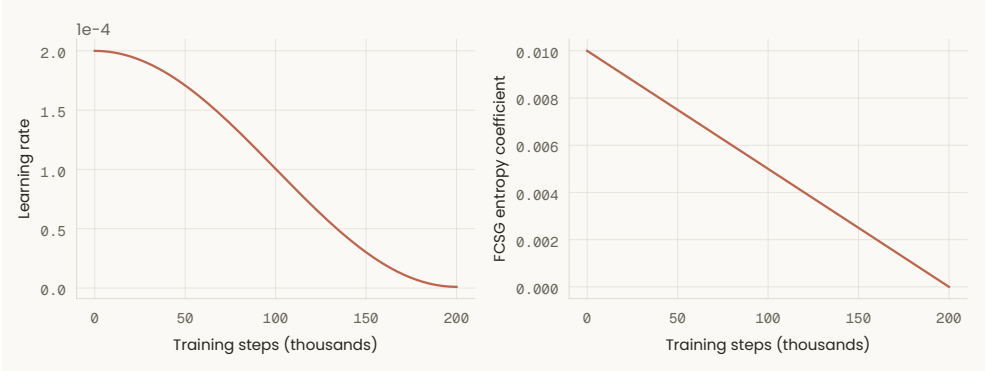


Figure 4.2: Learning-rate and entropy-coefficient schedules over 200,000 updates.

4.4 Evaluation Protocol

Final results use all 100 DIV2K validation images. Each image is degraded once with a seed equal to its index, so every model sees exactly the same degraded inputs. Inference is tiled: 256×256 tiles with 32 pixels of overlap (stride 224), with the last tile in each direction aligned to the image border and overlapping outputs averaged uniformly. Tiling keeps memory low on large images; Section 5.5 measures how much it costs in accuracy. Restored values are clipped to $[0, 1]$ before computing PSNR, SSIM and LPIPS as described in Section 2.4.

During training we monitored a different quantity: PSNR on 16 fixed 128×128 validation crops. The two are not comparable. For FCSG-Net the final crop PSNR is 30.59 dB while the full-image result is 27.24 dB. Crop numbers are used only for convergence curves.

4.5 Implementation Verification

Before any long training run we ran a fixed set of checks in a single notebook on the same Kaggle T4 (Python 3.12, PyTorch 2.10 [24], CUDA 12.8). The notebook stops at the first failure. Table 4.2 lists the checks and their measured values.

Table 4.2: Pre-training verification checks (Tesla T4, 25 September 2026 run).

Check	Criterion	Measured
FFDNet weights, CBSD68 σ=25	within 0.5 dB of 31.21	31.220 dB
Band reconstruction	$x_L + x_M + x_H = x$	max error 6.0×10^{-7}
Frequency units	same band at 128 and 256 px	passed
Parameter budget	< 5M	97,088
Compute budget	< 10 GFLOPs at 256^2	9.39
Memory, forward + backward	< 4 GB	0.60 GB
Shapes, gradients, odd sizes (AMP)	no errors	passed
One-image overfit, 200 steps	MSE < 0.01, $< \frac{1}{4}$ initial	ratio 0.18
Checkpoint restore	identical outputs	passed
Degradation reproducibility	byte-identical	passed

Two of these checks deserve a word. The band reconstruction assertion is a single line, yet it catches nearly every sign, shift and normalisation bug an FFT implementation can have. The overfit test asks the full model to memorise one image in 200 steps. A model that cannot do this has a bug, and finding that out in a few seconds is far cheaper than finding it after ten hours of training. FCSG-Net reduced the MSE from 0.0261 to 0.0046 (Figure 4.3). The memory figure was measured at batch size 1 and does not describe peak memory at the training batch size.

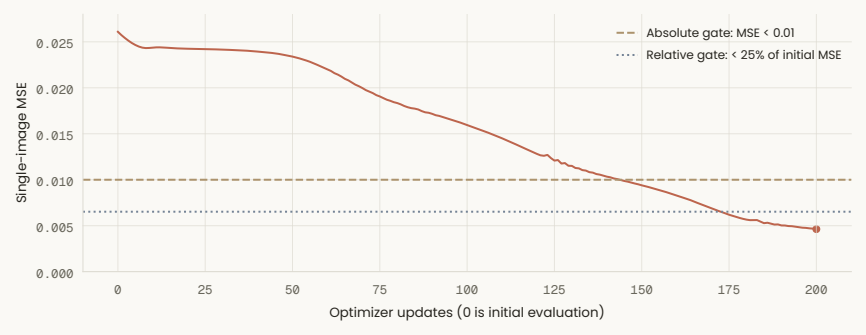


Figure 4.3: Single-image overfit test: MSE over 200 updates.

4.6 Baseline Reproduction

FFDNet. To check our FFDNet implementation and evaluation code against a published number, we loaded the author's pretrained colour weights [4] and evaluated them on all 68 CBSD68 images [25] with unclipped Gaussian noise at $\sigma = 25$, following the author's protocol. We obtained 31.220 dB, against 31.21 dB in the paper [3], a difference of 0.01 dB. This verifies the architecture, weight loading and evaluation path. It does not reproduce FFDNet's training, which is why all composite models are trained from scratch.

DnCNN. As an end-to-end test of our training loop we trained DnCNN on clipped Gaussian noise at $\sigma = 25$ before moving to the composite task. At checkpoint 286,000 it reached 32.65 dB on DIV2K validation and 31.04 dB on CBSD68 (Table 4.3). These figures use clipped noise and are not directly comparable with the unclipped protocol of the FFDNet paper; they confirm that the training and evaluation code work on a problem with a known order of magnitude.

Table 4.3: DnCNN on clipped Gaussian noise, $\sigma = 25$, checkpoint 286,000.

Dataset	Images	Input PSNR	Output PSNR
CBSD68	68	20.53 dB	31.04 dB
DIV2K validation	100	20.70 dB	32.65 dB

Chapter 5

Restoration Results and Computational Cost

This chapter reports how well the four trained models restore the 100 DIV2K validation images, how their quality relates to size and compute, and what they cost to run. Every model in this chapter was trained once, for 200,000 updates, under the protocol of Chapter 4.

5.1 Quantitative Comparison

Table 5.1 gives the main result. All models start from the same degraded inputs, with a mean PSNR of 20.99 dB.

Table 5.1: Composite restoration on 100 DIV2K validation images after 200,000 updates. Gain is output minus input PSNR (input 20.992 dB). GFLOPs at 256×256.

Method	PSNR↑	Gain	SSIM↑	LPIPS↓	Params	GFLOPs
DnCNN	27.831	6.838	0.7795	0.3242	558,403	73.43
FFDNet, blind	28.190	7.197	0.7899	0.3176	852,108	27.89
FFDNet, oracle†	28.240	7.247	0.7920	0.3162	852,108	27.89
FCSG-Net	27.243	6.250	0.7529	0.3961	97,088	9.39

†Given the true sampled noise level; an upper reference, not a fair competitor.

FCSG-Net improves the input by 6.25 dB, which shows that the network learns a substantial restoration, but it is the weakest of the four models on all three metrics. It trails DnCNN by 0.59 dB and blind FFDNet by 0.95 dB. The gap is larger in LPIPS than in PSNR: 0.396 against 0.318 for blind FFDNet, which suggests that FCSG-Net leaves more visible residue than its PSNR deficit alone would indicate.

The comparison looks different once size and compute are included (Table 5.2). FCSG-Net reaches within 0.6 dB of DnCNN with 82.6% fewer parameters and 87.2%

fewer reported FLOPs, and within 1 dB of both FFDNet variants with 88.6% fewer parameters and 66.3% fewer FLOPs.

Table 5.2: FCSG-Net relative to each baseline.

Compared with	PSNR difference	Fewer parameters	Fewer GFLOPs
DnCNN	−0.588 dB	82.6%	87.2%
FFDNet, blind	−0.947 dB	88.6%	66.3%
FFDNet, oracle	−0.997 dB	88.6%	66.3%

Two observations about the baselines are worth recording. Blind FFDNet is the strongest fair model, even though its noise map is a constant and so carries no information. Its advantage over DnCNN therefore comes from the architecture, most likely the half-resolution processing that gives each layer a larger effective receptive field, rather than from noise conditioning. Second, the oracle map adds only 0.05 dB over the blind constant. Knowing the noise level helps little when blur, resampling and JPEG artefacts are mixed in, because the map describes only one of the four corruptions.

Figure 5.1 plots the three metrics side by side.



Figure 5.1: Mean PSNR, SSIM and LPIPS over the 100 validation images. Each bar is one training run.

Each model was trained once with an unfixed seed, so we cannot separate the effect of the architecture from run-to-run variation. Differences of a few hundredths of a decibel, such as the one between the two FFDNet variants, should not be over-read. The gap between FCSG-Net and the baselines is large enough, and consistent enough across images (Section 5.2), that we do not expect a second seed to reverse it.

5.2 Per-Image Analysis

Averages can hide a model that wins on most images and loses badly on a few. Table 5.3 compares FCSG-Net with each baseline image by image.

Table 5.3: Per-image PSNR comparison of FCSG-Net against each baseline, 100 images.

Compared with	Wins / ties / losses	Mean difference	Range
DnCNN	6 / 1 / 93	−0.59 dB	−2.75 to +2.07 dB
FFDNet, blind	0 / 0 / 100	−0.95 dB	−3.12 to −0.04 dB
FFDNet, oracle	0 / 0 / 100	−1.00 dB	−3.48 to −0.07 dB

The deficit is systematic. Both FFDNet variants beat FCSG-Net on every one of the 100 images. Against DnCNN, FCSG-Net wins on six images, and on one of them by 2.07 dB, so the two convolutional designs fail in different places. Image 0801 is an example: DnCNN barely improves it (28.09 to 28.22 dB), while FCSG-Net reaches 29.32 dB. Conversely, DnCNN is the only model that makes some images worse; three of its outputs score below their inputs.

The hardest image for every model is 0828, whose degraded input scores only 17.10 dB. FCSG-Net reaches 17.93 dB on it and the baselines between 18.61 and 18.83 dB. Our experiment does not isolate which of the four sampled corruptions makes this image so difficult. Figure 5.2 shows the paired differences and the error distributions. The full table is in Appendix B.

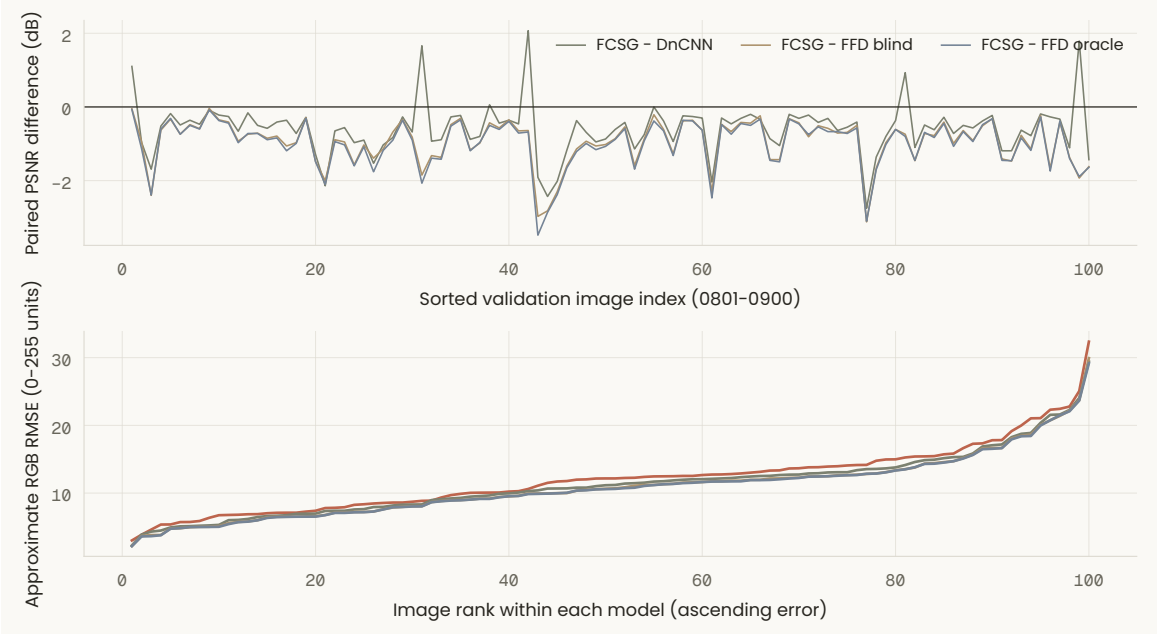


Figure 5.2: Left: per-image PSNR difference between FCSG-Net and each baseline. Right: sorted per-image RMSE for every model (in 8-bit units).

5.3 Qualitative Results

Figures 5.3 and 5.4 show the top-left 320×320 region of validation images 0801 and 0802 for each model. Every model receives the identical degraded input. PSNR labels refer to the whole image, not the displayed region.

All four models remove most of the noise and blocking. None of them recovers the fine texture that blur and downsampling destroyed, which is expected: those details are not in the input, and a model trained with pixel losses predicts a smooth average rather than inventing texture. Differences between models at this scale are subtle, and the saved panels do not include difference images, so we rely on the quantitative results above rather than on visual judgement.

5.4 Training Convergence

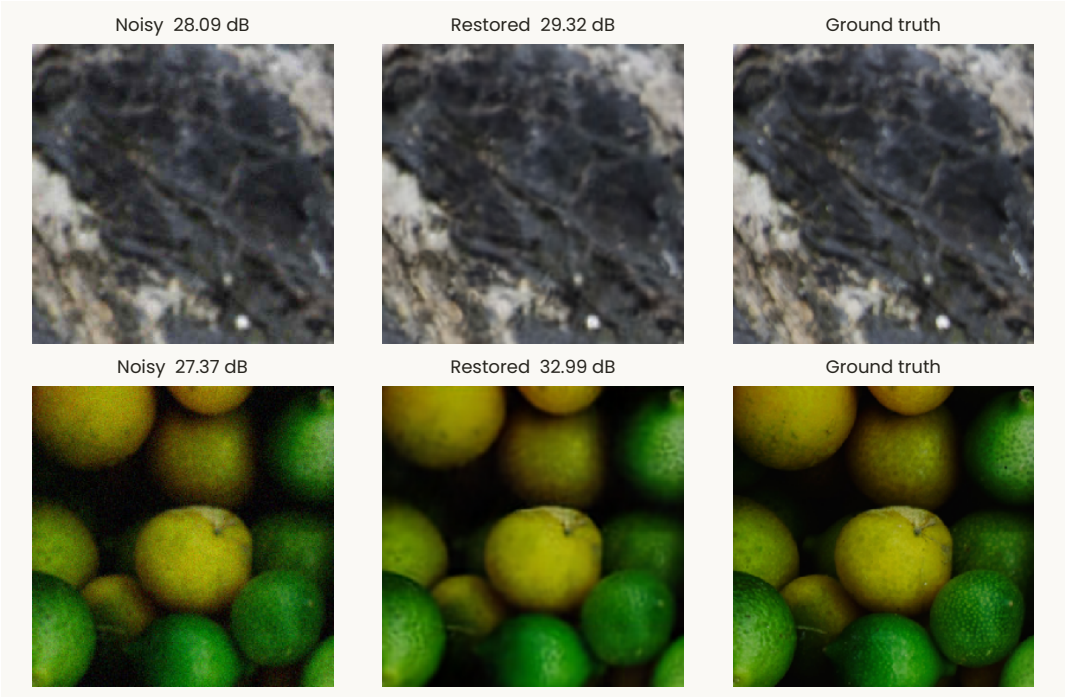
Figure 5.5 plots validation PSNR on the 16 fixed crops against both updates and wall-clock training time. All four models are close to a plateau by the end of training. FCSG-Net finishes at 30.59 dB on the crops, below DnCNN (31.66), blind FFDNet (31.97) and oracle FFDNet (32.05), the same ordering as the full evaluation. Because the curves have flattened, we have no evidence that simply training FCSG-Net for longer with the same configuration would close the gap.

The wall-clock panel exposes a cost that the update count hides. FCSG-Net needed 10.06 hours for 200,000 updates, against 6.66 for DnCNN and about two for FFDNet. A separate throughput test, excluding warm-up, validation and checkpointing, measured 5.84 steps per second for FCSG-Net, 9.41 for DnCNN and 26.67 for FFDNet. Matching the number of updates therefore did not match the GPU time, and FCSG-Net received the most of it.

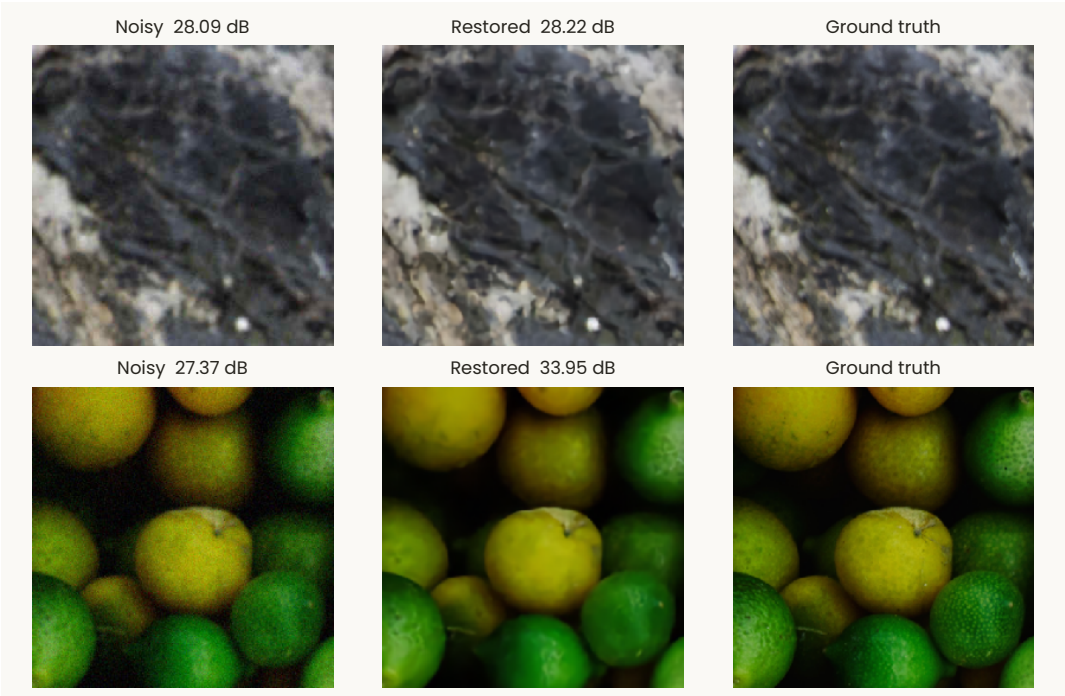
5.5 Latency, Memory and Tiling

Latency. The motivation for a small model is that it should be cheap to run. Table 5.4 shows that, on the GPU we used, this did not hold. Measurements are median wall-clock times for one FP32 forward pass at batch size 1 on a Tesla T4, with TF32 disabled, after 20 warm-up passes and over 50 timed repetitions.

FCSG-Net is the slowest model at every size, slightly behind DnCNN and three times slower than FFDNet, despite needing an eighth of DnCNN's reported FLOPs and a third of FFDNet's. It also uses the most memory. We did not profile individual operators, so the following explanation is our best reading of the architecture rather than a measurement. Each forward pass runs nine expert passes, all of them in full, because zero-weighted experts are not skipped. Depthwise convolutions perform very

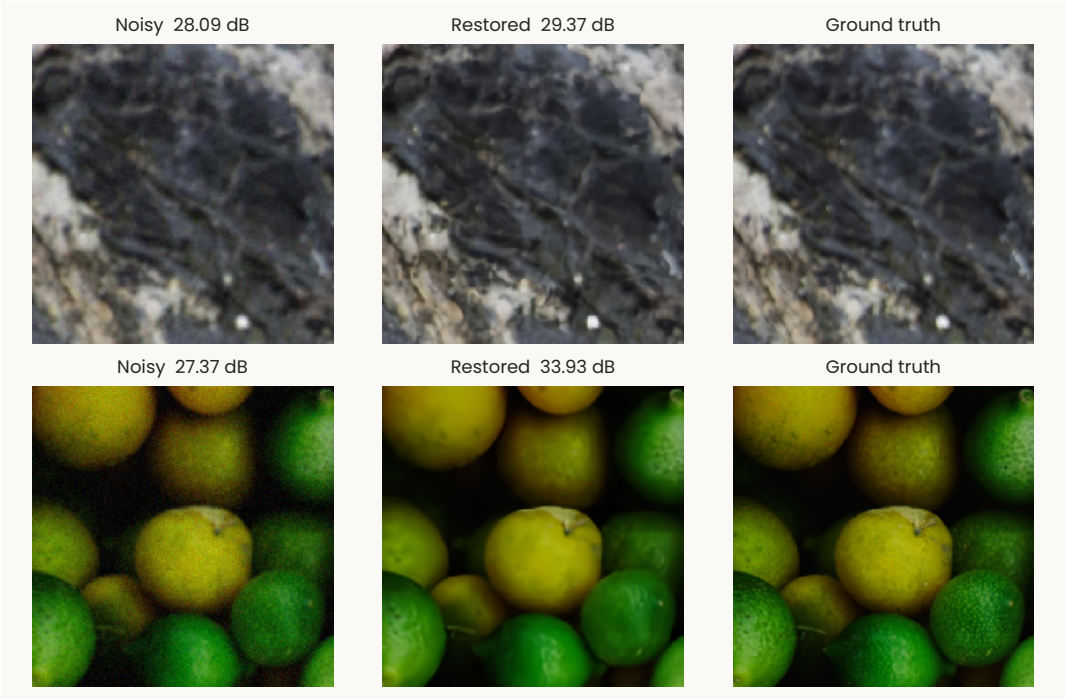


(a) FCSG-Net

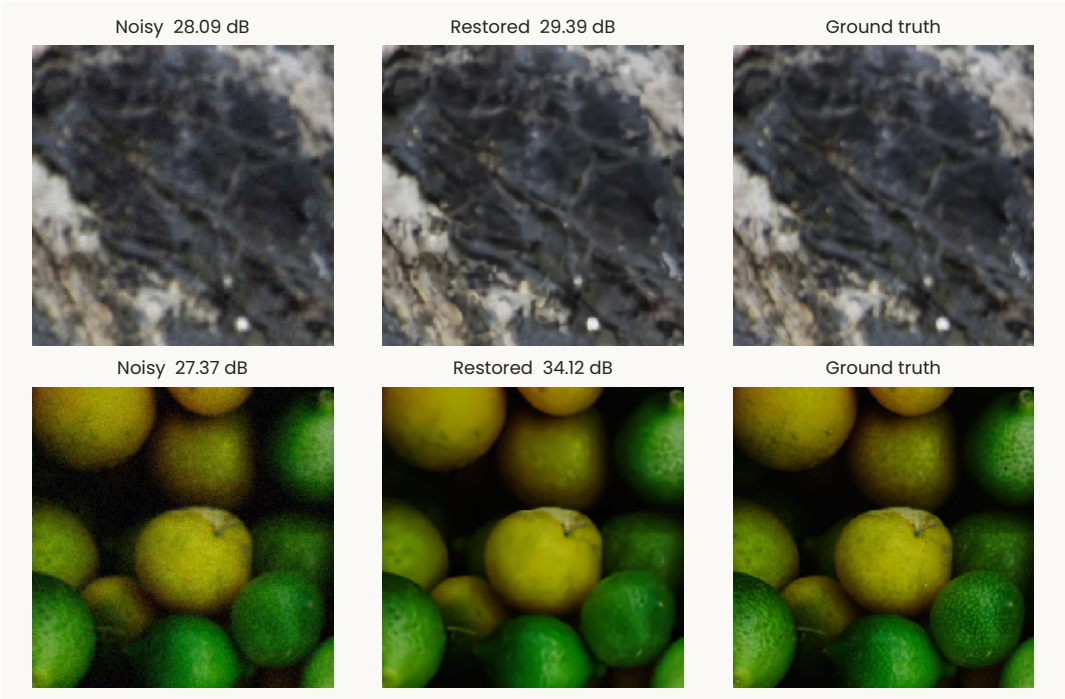


(b) DnCNN

Figure 5.3: Degraded input, restoration and clean target for FCSG-Net and DnCNN.



(a) FFDNet, blind (σ map fixed at 27.5/255)



(b) FFDNet, oracle (true σ map)

Figure 5.4: Degraded input, restoration and clean target for the two FFDNet variants.



Figure 5.5: Validation PSNR on 16 fixed crops against updates and training time. Crop PSNR is not comparable with the full-image results.

Table 5.4: Median forward latency (ms) by input size and peak inference memory at 256×256 (Tesla T4, FP32, batch 1).

Method	128^2	256^2	512^2	Peak MB at 256^2
FCSG-Net	5.10	19.40	75.59	51.3
DnCNN	4.22	18.04	72.47	37.8
FFDNet, blind	1.69	6.30	24.12	20.2
FFDNet, oracle	1.69	6.35	24.62	20.2

little arithmetic per byte they read, so on a GPU their running time is governed by memory traffic, not by the FLOP count. The network is also made of many small layers, so kernel launch overhead is a large fraction of the total, and the FFT and inverse FFT add work that the FLOP estimate counts only approximately. FFDNet, by contrast, runs one stack of dense 3×3 convolutions at half resolution, which is exactly the kind of workload GPUs are built for.

The practical lesson is that the FLOP budget we set at the start of the project was the wrong proxy for speed on this hardware. Figure 5.6 plots quality against both measures. Against FLOPs, FCSG-Net sits far to the left of the other models. Against latency it sits to the right of FFDNet with lower quality, so on a T4 it is dominated. Whether the picture changes on a CPU or a mobile accelerator, where depthwise convolutions are better suited, remains to be tested.

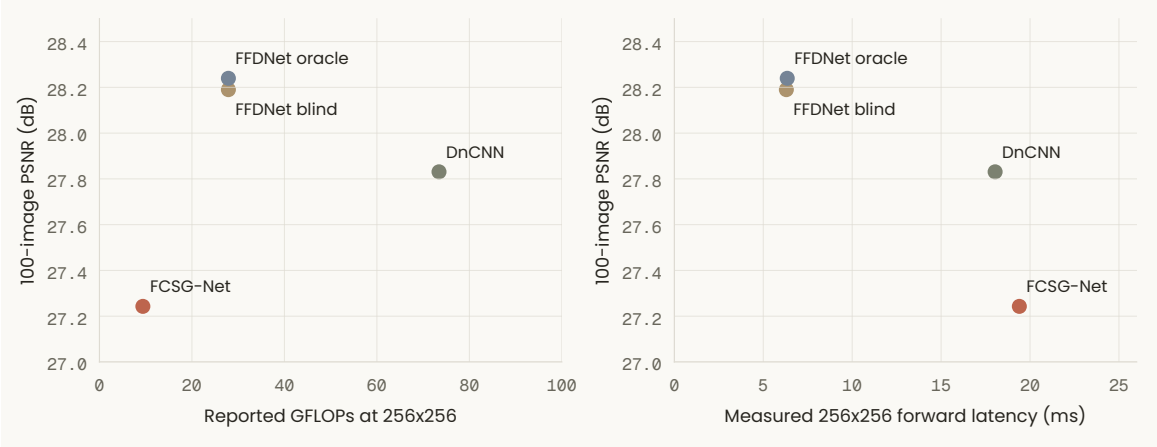


Figure 5.6: Validation PSNR against reported GFLOPs (left) and measured 256×256 forward latency (right).

Tiling. Tiled inference changes what each model sees: the FFT, the gate's pooling windows and the global pooling in the fusion block all operate on a 256×256 tile rather than the whole image. To measure the effect we evaluated ten fixed validation images (0801, 0812, ..., 0900) with tile sizes of 128, 256 and 512 pixels and a fixed 32-pixel overlap (Table 5.5).

Table 5.5: Mean PSNR (dB) on ten validation images for three tile sizes, 32-pixel overlap.

Method	Tile 128	Tile 256	Tile 512
FCSG-Net	26.622	26.659	26.672
DnCNN	27.330	27.341	27.343
FFDNet, blind	27.696	27.714	27.721
FFDNet, oracle	27.730	27.748	27.756

FCSG-Net is the most sensitive to tile size, as expected for a model with global operations, but the effect is small: going from 256 to 512 pixel tiles gains 0.013 dB, against 0.002 dB for DnCNN and about 0.008 dB for FFDNet. This is two orders of magnitude smaller than the 0.6 to 1.0 dB gap in Table 5.1, so tiling does not explain the deficit. The ten-image means are lower than the 100-image means because the subset happens to contain harder images; they should be compared only with each other. Figure 5.7 shows the paired differences.

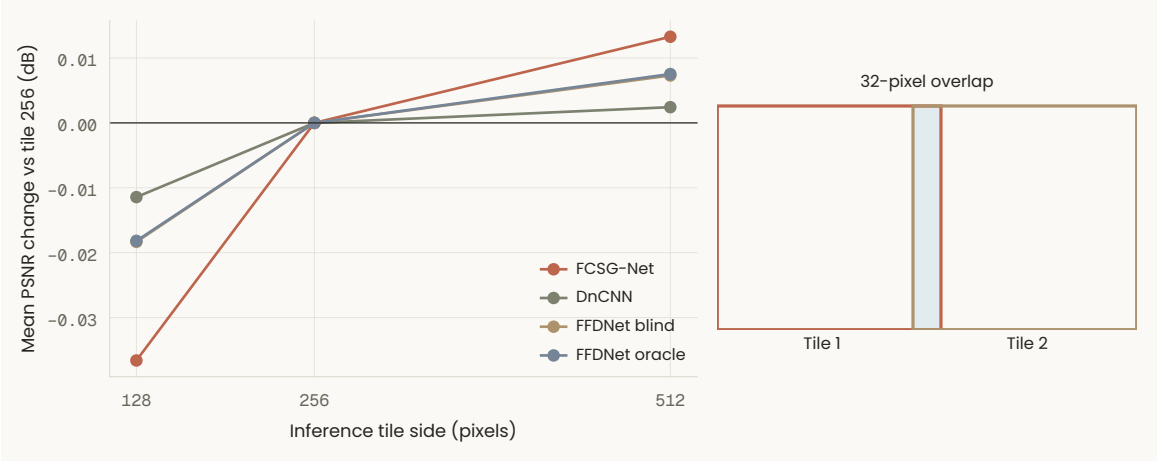


Figure 5.7: Tile-size sensitivity: PSNR change relative to 256-pixel tiles, and the tile and overlap geometry.

Chapter 6

Routing Analysis

Chapter 5 asked how well FCSG-Net restores images. This chapter asks what its gate actually does. All measurements here use the final 200,000-update checkpoint from Chapter 5. They were taken on ten fixed validation images (0801, 0812, . . . , 0900), each degraded with eight different random seeds, giving 80 samples. For each sample we take the 256×256 centre crop after degrading the whole image. The ten source images are the independent units; the eight seeds per image vary only the degradation.

6.1 Training Dynamics of the Gate

The entropy term in the loss is meant to keep all three experts in use early on and then fade out so that the gate can commit. Figure 6.1 shows the dense routing entropy during training, summed over the three bands. Its maximum possible value is $3 \ln 3 = 3.296$ nats, reached when every expert gets equal probability everywhere.



Figure 6.1: Dense routing entropy summed over bands during training. The maximum is $3 \ln 3 \approx 3.296$ nats.

At the end of training the dense probabilities remain close to uniform. On the diagnostic samples the mean dense entropy per band is 1.089, 1.096 and 1.097 nats for the low, mid and high bands, against a maximum of $\ln 3 = 1.099$. The sparse top-2 weights are similarly close to an even split, at 0.684, 0.691 and 0.692 nats against $\ln 2 = 0.693$. Taken alone, these numbers would suggest that the gate learned almost

nothing. The rest of this chapter shows why that conclusion would be wrong: small differences between logits still change *which* two experts are selected, and the selection turns out to matter.

6.2 Spatial Routing Statistics

Figure 6.2 summarises the routing over the 80 samples. Because two experts are selected at each pixel, the selection fractions in each band add up to two.



Figure 6.2: Mean sparse weight and selection fraction for each expert and band over 80 samples.

The clearest pattern is in the high band, where the 3×3 expert is selected at 99.98% of pixels. The other slot is shared between the larger experts, which still carry substantial weight on average (about 0.26 for the 7×7 expert and 0.22 for the 5×5). The high band is therefore not handed entirely to the small kernel; rather, the small kernel is always present and one of the larger kernels is mixed in depending on the location. This agrees with the intuition behind the design, that fine-scale content is best handled with a small support, though the routing data alone cannot show that the 3×3 expert is doing anything specific to high frequencies.

Figure 6.3 shows the routing maps for one image. The maps follow image structure: selection changes along object boundaries and between smooth and textured regions. Because a top-2 choice switches abruptly when two logits exchange rank, smooth underlying probabilities can still produce sharp boundaries between selected pairs.

These maps describe the gate's output. They do not establish what each expert contributes to a particular restored structure, which would require attribution experiments we did not run.

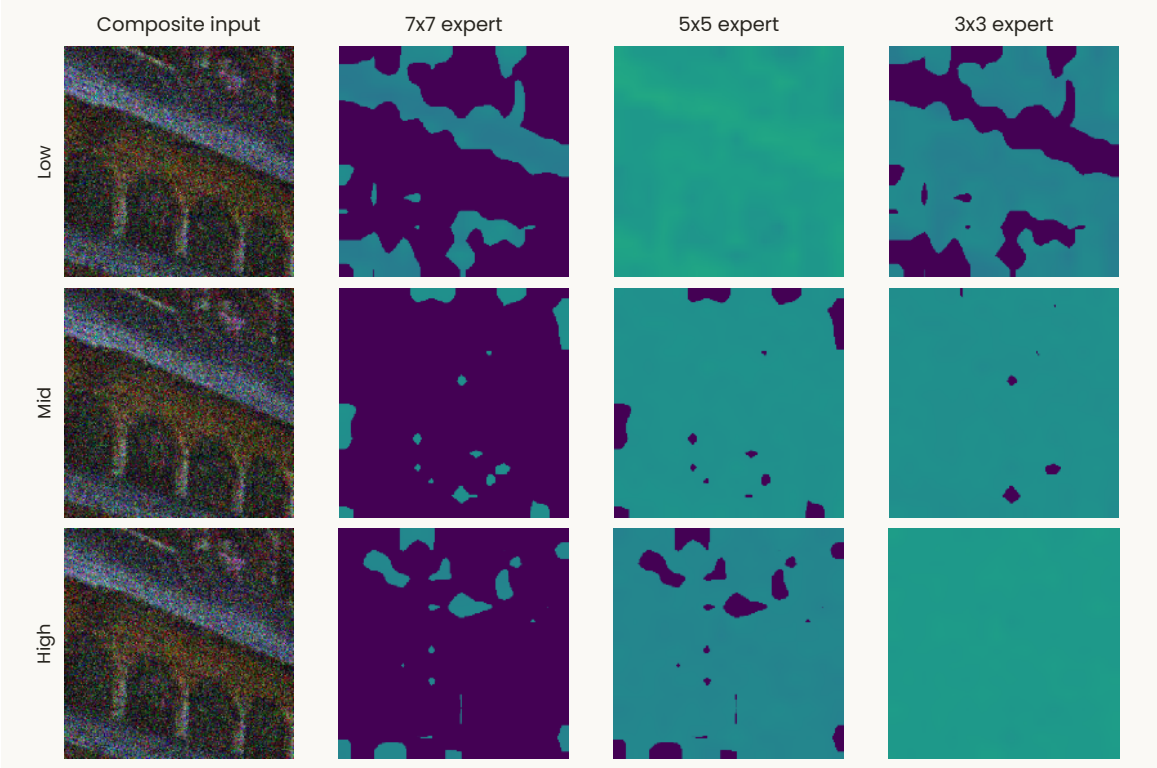


Figure 6.3: Expert weights for validation image 0834 (seed 33). Rows: low, mid and high band. Columns: degraded crop and the weights of the 7×7, 5×5 and 3×3 experts, on a common 0 to 1 scale.

6.3 Routing Versus Degradation Parameters

Because every sample records its degradation parameters, we can ask whether the routing responds to them. For each image we subtract its eight-sample mean from both the mean routing weights and the degradation parameters, so that the comparison is within images and not driven by differences in image content. We then compute Pearson correlations between the nine mean expert weights and the four parameters.

Several of these correlations are strong. The mean weight of the 3×3 expert in the high band correlates at $r = +0.81$ with the noise level: the noisier the input, the more the gate leans on the small kernel for high frequencies. In the low band, the 5×5 expert's weight correlates at $+0.80$ with noise and the 7×7 expert's at -0.73, so with more noise the gate shifts low-band weight from the largest kernel to the medium one. The mid band follows the same direction, with the 3×3 weight at $+0.77$. The six strongest correlations are all with the noise level. The strongest with blur is $+0.42$ and with JPEG quality -0.39, so within this sample the gate responds mainly to noise. Figure 6.4 shows the full correlation matrix next to the intervention results of the next section.

We treat these results as exploratory. There are 36 correlations computed on ten

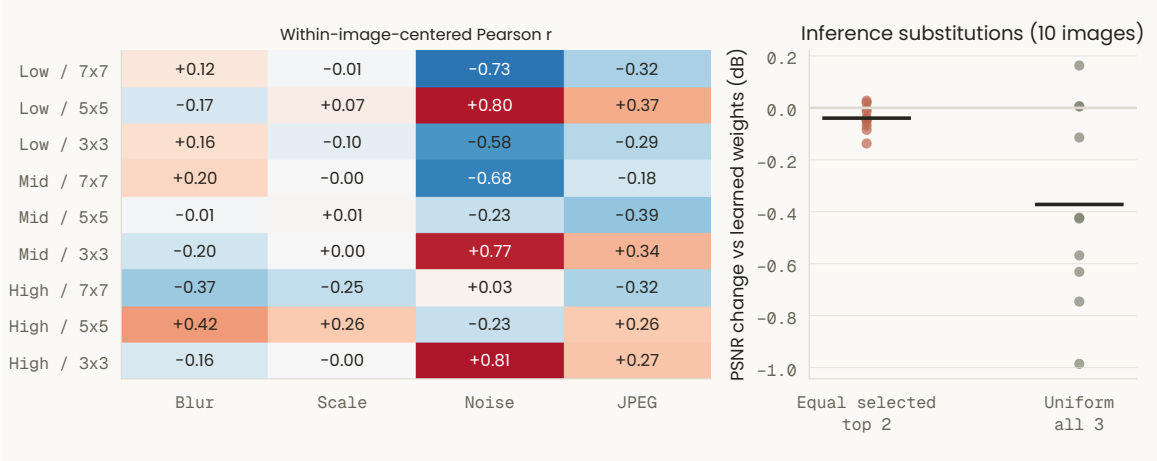


Figure 6.4: Left: within-image Pearson correlations between mean routing weights and degradation parameters. Right: per-image PSNR change under the inference interventions of Table 6.1.

source images, without correction for multiple comparisons and without image-level confidence intervals. The four parameters also vary together in every sample, so a correlation with noise cannot be fully separated from the other corruptions. A controlled sweep that varies one parameter at a time would be needed to make a causal statement. What the data do show is that the gate's output is not constant: it moves with the degradation in a consistent direction across images.

6.4 Inference-Time Interventions

Correlation shows that the gate responds to its input. It does not show that the network needs that response. To test this we kept the trained weights fixed and replaced the gate's output at inference time in two ways:

- *Equal pair weights*: keep the two experts the gate selects at each pixel, but give each a weight of one half.
- *Uniform weights*: ignore the gate and give all three experts a weight of one third everywhere.

Table 6.1 reports the change in PSNR on the ten images.

Table 6.1: PSNR change when the trained gate is overridden at inference (10 images).

Intervention	Mean ΔPSNR	Min / max
Equal weights on the selected pair	−0.040 dB	−0.137 / +0.028
Uniform weights over all three experts	−0.372 dB	−0.986 / +0.163

The two interventions separate two parts of the gate's decision. Flattening the weights within the selected pair costs only 0.04 dB on average, so the exact mixing proportions matter little. Removing the selection itself costs 0.37 dB on average and up to almost 1 dB on one image. The trained network therefore depends mainly on *which* experts are combined at each pixel, and much less on how they are weighted. This also explains how the gate can matter while its entropy stays near the maximum: a near-uniform distribution over three experts still leaves one of them consistently excluded.

A limitation of this test is that it measures dependence on the gate of *this* trained network. Its experts and fusion stage were trained alongside the gate and adapted to its choices, so overriding the gate puts the model in a situation it never saw. The result shows that the gate is doing real work in the trained model. It does not show that a model trained from the start without a gate would be worse. Answering that requires a matched training ablation, which we list as future work (Section 7.3).

6.5 Discussion

Putting Chapters 5 and 6 together gives a mixed but coherent picture. The routing mechanism works in the sense we set out to test. The gate produces spatially varying maps, its choices track the noise level in a consistent direction, and the trained model loses up to a decibel when its choices are overridden. These are the behaviours the design was meant to produce.

What the routing does not do is make the model competitive. With fewer than a hundred thousand parameters, FCSG-Net ends up 0.6 to 1.0 dB behind baselines that are six to nine times larger, and on a GPU it is not faster than any of them. The likely reason for the quality gap is simply capacity. Even the largest expert has under 40,000 parameters, and much of the network's width is spent on processing three bands that sum to the original image. Blind FFDNet, the strongest fair model, gets its advantage from a plain and dense architecture, not from extra information.

A useful way to read the result is that frequency-band routing is a cheap way to make a small model's behaviour *interpretable*: we can see where each kernel size is used and relate it to the degradation. Within the budget we chose, it is not by itself a way to make the model better or faster.

Chapter 7

Conclusion and Future Work

7.1 Summary of Findings

We set out to test whether a very small restoration network could split an image into frequency bands and route each band, pixel by pixel, to convolutional experts of different kernel sizes, and whether the learned routing would be meaningful. We return to the four objectives of Section 1.3.

O1, build and verify. FCSG-Net has 97,088 parameters and costs 9.39 reported GFLOPs at 256×256, well inside the limits of five million and ten. It passed every verification check before training: exact band reconstruction, consistent frequency units across image sizes, gradient flow, mixed-precision execution, single-image overfitting and exact checkpoint restoration (Section 4.5).

O2, trustworthy baselines. Our FFDNet implementation reproduces the published CBSD68 result to within 0.01 dB using the author's weights, and our DnCNN training loop reaches 32.65 dB on DIV2K for Gaussian denoising (Section 4.6).

O3, matched comparison. Under one composite degradation and one training protocol, FCSG-Net reaches 27.24 dB PSNR, 0.753 SSIM and 0.396 LPIPS on 100 DIV2K validation images. This is 0.59 dB below DnCNN and 0.95 dB below blind FFDNet, and it loses to FFDNet on every one of the 100 images. It does so with 82.6% fewer parameters than DnCNN and 88.6% fewer than FFDNet. Its low FLOP count, however, did not make it fast: on a Tesla T4 it is the slowest of the four models (Chapter 5).

O4, the routing. The learned gate varies across the image, selects the 3×3 expert almost everywhere in the high band, and shifts its weights with the noise level, with

within-image correlations up to $r = +0.81$. Overriding its selections at inference costs 0.37 dB on average, while flattening its weights within the selected pair costs only 0.04 dB, so the trained model depends on which experts are chosen more than on how they are mixed (Chapter 6).

In short, frequency-band routing produced a small model whose behaviour can be inspected and related to the degradation, but within our budget it did not produce a model that is better or faster than simple convolutional baselines.

7.2 Limitations

Several limitations bound how far these results can be taken.

Each model was trained once with an unfixed seed, so we have no estimate of run-to-run variance. The consistency of the gap across images makes it unlikely to be an accident of one seed, but small differences, such as between the two FFDNet variants, cannot be interpreted. All results are on the DIV2K validation split, which was also used to monitor training; there is no untouched test set.

The degradation is synthetic. Real archival photographs suffer from fading, scratches, dust, colour casts and scanner artefacts that our pipeline does not model, and we did not evaluate on real archival images or ask people to judge the results. The good performance of any of these models on our data says nothing yet about real archives.

The routing analysis uses ten source images. Its correlations are exploratory, the four degradation parameters vary together, and there is no correction for the number of comparisons. The inference interventions measure the trained model's dependence on its gate, not the value of having a gate during training.

The cost measurements cover one GPU, the Tesla T4, at batch size 1 in FP32. We did not profile individual operators, so our explanation of why FCSG-Net is slow is a reasoned account rather than a measurement. We did not measure latency on CPUs or mobile hardware, nor peak training memory at batch 16.

Finally, FCSG-Net computes all nine expert passes regardless of the gate, so its sparse routing does not save computation.

7.3 Future Work

The most important next step is a matched training ablation: train FCSG-Net and an otherwise identical model with fixed uniform weights from the same initialisation and seed, and compare them on the full validation set, preferably with several seeds each. This is the only experiment that can show whether learning a router helps,

rather than whether a trained model depends on its router. Related ablations would replace the three kernel sizes with three identical 3×3 experts, replace the spatial gate with a single per-image gate, replace SE fusion with plain concatenation, and replace the soft masks with hard ones. Each of these isolates one design decision of Chapter 3.

The second direction is to turn sparse weights into sparse computation. Since the trained model mostly cares about which two experts are chosen, skipping the third expert in tiles where it is never selected could reduce the running cost by up to a third. This should be preceded by an operator-level profile, because if latency is dominated by memory traffic and launch overhead, removing arithmetic may not help much. Measuring latency on a CPU and on a mobile accelerator would show whether the model's low FLOP count is worth more on hardware that suits depthwise convolutions.

Third, the capacity gap could be addressed directly by widening the experts within the parameter budget, which is about fifty times larger than what the model uses. The compute budget is nearly exhausted, so this would have to go together with cheaper expert evaluation, for example by sharing one unshuffled feature map across bands. Letting the gate see a larger context than a 16×16 window may also help.

Finally, any claim about archival restoration needs real archival photographs, a controlled degradation sweep to separate the effect of each corruption on the routing, and a held-out test set that was never used during development.

References

[1] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142–3155, 2017.

[2] S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan, and M.-H. Yang, "Restormer: Efficient transformer for high-resolution image restoration," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

[3] K. Zhang, W. Zuo, and L. Zhang, "FFDNet: Toward a fast and flexible solution for CNN-based image denoising," *IEEE Transactions on Image Processing*, vol. 27, no. 9, pp. 4608–4622, 2018.

[4] K. Zhang, "KAIR: Image restoration toolbox," https://github.com/cszn/KAIR, fFDNet inference script and pretrained colour weights.

[5] L. Chen, X. Chu, X. Zhang, and J. Sun, "Simple baselines for image restoration," in *European Conference on Computer Vision*, 2022.

[6] P. Liu, H. Zhang, K. Zhang, L. Lin, and W. Zuo, "Multi-level wavelet-CNN for image restoration," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2018.

[7] L. Chi, B. Jiang, and Y. Mu, "Fast Fourier convolution," in *Advances in Neural Information Processing Systems*, 2020.

[8] Y. Cui, Y. Tao, Z. Bing, W. Ren, X. Gao, X. Cao, K. Huang, and A. Knoll, "Selective frequency network for image restoration," in *International Conference on Learning Representations*, 2023.

[9] Y. Cui, W. Ren, X. Cao, and A. Knoll, "Image restoration via frequency selection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[10] L. Kong, J. Dong, J. Ge, M. Li, and J. Pan, "Efficient frequency domain-based transformers for high-quality image deblurring," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

[11] N. Rahaman *et al.*, "On the spectral bias of neural networks," in *International Conference on Machine Learning*, 2019.

[12] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[13] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[14] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.

[15] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[16] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.

[17] W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang, "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

[18] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition*, 2017.

[19] P. Charbonnier, L. Blanc-Féraud, G. Aubert, and M. Barlaud, "Two deterministic half-quadratic regularization algorithms for computed imaging," in *Proc. IEEE International Conference on Image Processing*, 1994.

[20] E. Agustsson and R. Timofte, "NTIRE 2017 challenge on single image super-resolution: Dataset and study," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2017.

[21] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[22] ——, "SGDR: Stochastic gradient descent with warm restarts," in *International Conference on Learning Representations*, 2017.

[23] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *International Conference on Learning Representations*, 2018.

[24] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.

[25] D. Martin, C. Fowlkes, D. Tal, and J. Malik, "A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics," in *Proc. IEEE International Conference on Computer Vision*, 2001.

Appendix A

Reproducibility Record

A.1 Code and Environment

All code, configuration files, logs and evaluation outputs are in the project repository at `https://github.com/whynotramaa/fcsg-capstone`. Model code lives in `models/` and `src/fcsg_net/`, training in `training/train.py`, and evaluation and plotting in `evaluation/`. Every model is selected by a configuration file in `configs/`; the four composite runs use `fcsg.toml`, `dncnn_composite.toml`, `ffdnet_blind.toml` and `ffdnet_composite.toml`, which differ only in the model and noise map.

Training, verification and diagnostics ran on Kaggle with an NVIDIA Tesla T4, Python 3.12.13, PyTorch 2.10.0 with CUDA 12.8, NumPy 2.0.2 and Pillow 11.3.0. Local machines were used only to edit and package code.

A.2 Where Each Result Comes From

Table A.1: Source files for the results in this report.

Result	File
Verification checks (Table 4.2)	`results/milestone_run_20260925/`
Gaussian DnCNN (Table 4.3)	`results/benchmark.csv`
Composite results (Table 5.1)	`results/phase3/*/evaluation.json`
Per-image results (Appendix B)	`results/phase3/per_image_scores.csv`
Training curves	`results/phase3/*/train_log.csv`
Latency, tiling, routing	`results/diagnostics/`
Figures	`output/pdf/fcsg-report-assets/`

A.3 Artefact Identifiers

Each Kaggle run packaged its source code into a bundle whose SHA-256 hash was recorded, so the exact code behind every result can be identified even after later edits. Table A.2 lists the first sixteen hexadecimal digits of each hash; the full values are in the source manifests.

Table A.2: SHA-256 identifiers (first 16 hex digits) of evaluated artefacts.

Artefact	SHA-256 prefix
Composite training source bundle	`85d6e16f72ead013`
Diagnostics source bundle	`d473bc10b0ad0c1e`
5,000-pair dataset manifest	`7ce7f310bdb4d2d5`
FCSG-Net checkpoint	`8db0bb0f3c251ddc`
DnCNN checkpoint	`afb0abfa022ede04`
FFDNet blind checkpoint	`981560c18cbea134`
FFDNet oracle checkpoint	`ef9560e8ed1ebae0`

A.4 How to Reproduce

1. Attach the DIV2K dataset to a Kaggle notebook with a GPU and run `notebooks/checks.ipynb`, then the bundled milestone notebook `notebooks/phase1_phase2.ipynb`. Both stop at the first failed check.

2. Build and push one training notebook per configuration with `python notebooks/build_phase3.py fcsg ffdnet_blind ffdnet_composite dncnn_composite`. Runs longer than one session resume from their last checkpoint when rebuilt with `--resume`.

3. At 200,000 updates each run evaluates the 100 validation images and writes `evaluation.json` and `status.json`.

4. Build the diagnostic notebook with `python notebooks/build_diagnostics.py` to measure latency, tiling and routing, and draw figures with `python evaluation/plots.py`.

Appendix B

Per-Image Results

Table B.1 lists the input PSNR and the output PSNR of every model for each of the 100 DIV2K validation images, in dB. The best output for each image is in bold. Values come from `results/phase3/per_image_scores.csv` and are rounded to two decimal places, so their means can differ from Table 5.1 in the last digit. The table is generated by `thesis/tools/per_image_table.py`.

Table B.1: Per-image PSNR (dB) on DIV2K validation.

Image	Input	DnCNN	FFDNet blind	FFDNet oracle	FCSG-Net
0801	28.09	28.22	29.37	**29.39**	29.32
0802	27.37	33.95	33.93	**34.12**	32.99
0803	15.71	33.77	**34.48**	34.47	32.08
0804	16.16	26.95	**27.06**	27.03	26.43
0805	14.82	25.82	**25.97**	25.95	25.64
0806	18.14	26.57	**26.82**	**26.82**	26.08
0807	18.11	20.51	20.63	**20.65**	20.15
0808	16.25	25.80	25.92	**25.93**	25.33
0809	22.81	**31.57**	31.52	31.56	31.48
0810	16.54	25.48	25.61	**25.63**	25.26
0811	15.96	26.43	26.58	**26.61**	26.17
0812	17.27	28.60	28.87	**28.91**	27.94
0813	28.36	31.28	**31.86**	31.84	31.12
0814	15.68	26.52	26.73	**26.74**	26.02
0815	17.65	29.74	30.01	**30.05**	29.16
0816	25.01	30.11	30.49	**30.54**	29.70
0817	29.35	31.27	31.97	**32.10**	30.91
0818	17.80	26.89	27.14	**27.16**	26.17
0819	25.06	28.17	28.18	**28.20**	27.89
0820	20.31	26.03	26.22	**26.23**	24.74

Image	Input	DnCNN	FFDNet blind	FFDNet oracle	FCSG-Net
0821	27.80	**31.93**	31.79	31.89	29.79
0822	17.92	27.18	27.42	**27.47**	26.53
0823	26.21	27.79	28.18	**28.26**	27.23
0824	25.07	27.88	28.48	**28.51**	26.91
0825	21.15	27.46	27.59	**27.64**	26.56
0826	25.15	27.29	27.15	**27.52**	25.76
0827	30.95	34.00	34.11	**34.15**	32.97
0828	17.10	18.76	18.61	**18.83**	17.93
0829	15.75	25.43	25.50	**25.52**	25.16
0830	17.82	26.30	26.44	**26.52**	25.62
0831	29.44	29.73	33.24	**33.46**	31.39
0832	20.16	27.15	27.54	**27.61**	26.22
0833	25.07	33.64	34.12	**34.17**	32.75
0834	16.05	24.65	24.86	**24.90**	24.38
0835	23.60	24.43	24.51	**24.55**	24.20
0836	15.53	25.50	25.79	**25.81**	24.62
0837	15.65	25.92	**26.10**	26.08	25.12
0838	29.15	36.36	36.85	**36.92**	36.42
0839	17.69	28.06	28.19	**28.23**	27.62
0840	27.16	29.08	29.07	**29.11**	28.72
0841	17.67	25.90	26.09	**26.15**	25.44
0842	26.95	26.16	28.87	**28.91**	28.23
0843	20.44	35.45	36.51	**37.02**	33.54
0844	36.25	41.00	41.39	**41.43**	38.57
0845	23.08	26.39	26.67	**26.75**	24.37
0846	16.09	24.52	24.95	**25.00**	23.35
0847	23.40	25.60	26.37	**26.44**	25.23
0848	21.99	25.80	26.04	**26.10**	25.11
0849	19.78	22.61	22.72	**22.82**	21.66
0850	21.91	28.95	29.10	**29.15**	28.08
0851	20.99	26.99	27.24	**27.26**	26.38
0852	26.22	28.52	28.64	**28.69**	28.10
0853	19.11	31.31	31.76	**31.86**	30.17
0854	22.68	26.73	26.85	**26.90**	25.98
0855	28.26	30.76	30.97	**31.14**	30.76
0856	21.16	23.49	23.72	**23.76**	23.11
0857	13.93	32.51	32.84	**32.89**	31.57
0858	16.78	26.08	**26.22**	26.20	25.84
0859	23.45	26.47	26.57	**26.59**	26.21
0860	20.09	21.46	**21.79**	**21.79**	21.16

Image	Input	DnCNN	FFDNet blind	FFDNet oracle	FCSG-Net
0861	23.50	26.18	26.43	**26.61**	24.14
0862	16.49	31.67	31.84	**31.85**	31.37
0863	17.92	30.76	30.97	**31.04**	30.30
0864	26.45	29.89	30.00	**30.03**	29.58
0865	25.90	26.64	26.88	**26.94**	26.44
0866	20.34	**26.77**	26.67	26.76	26.43
0867	25.37	27.57	28.14	**28.16**	26.71
0868	19.78	32.57	32.95	**33.01**	31.52
0869	16.42	23.58	23.70	**23.71**	23.38
0870	15.03	26.23	26.35	**26.38**	25.92
0871	26.18	30.48	**31.07**	31.01	30.26
0872	16.17	24.13	24.22	**24.25**	23.71
0873	22.14	23.43	23.70	**23.79**	23.12
0874	22.38	30.14	**30.21**	30.17	29.50
0875	15.92	22.67	22.81	**22.84**	22.12
0876	18.52	22.90	22.99	**23.06**	22.49
0877	17.99	33.85	**34.22**	34.20	31.10
0878	22.03	30.80	31.10	**31.14**	29.44
0879	21.48	27.56	27.73	**27.78**	26.76
0880	14.46	28.77	**29.01**	**29.01**	28.40
0881	25.37	25.34	27.01	**27.07**	26.27
0882	15.19	30.54	**30.90**	30.89	29.44
0883	21.42	25.12	**25.35**	25.32	24.63
0884	18.18	26.04	26.20	**26.25**	25.42
0885	15.06	21.95	22.08	**22.12**	21.67
0886	19.79	34.28	34.55	**34.63**	33.56
0887	20.01	24.85	24.99	**25.02**	24.35
0888	25.69	29.76	30.10	**30.13**	29.19
0889	17.31	28.44	**28.57**	28.54	28.06
0890	17.79	24.69	**24.78**	**24.78**	24.46
0891	20.27	26.51	26.73	**26.77**	25.32
0892	15.73	27.55	**27.83**	27.82	26.36
0893	24.56	31.73	31.87	**31.94**	31.10
0894	22.61	28.85	29.21	**29.25**	28.07
0895	15.54	21.17	21.21	**21.25**	20.98
0896	28.23	35.13	36.53	**36.60**	34.86
0897	17.12	21.44	**21.52**	21.49	21.11
0898	19.19	32.31	32.57	**32.60**	31.20
0899	25.23	24.41	**28.13**	28.09	26.20
0900	19.34	27.46	27.65	**27.67**	26.03

Image	Input	DnCNN	FFDNet blind	FFDNet oracle	FCSG-Net